

Recall-Aware ML-IDS Platform

Complete Codebase Architecture, File-by-File Technical Deep Dive & Defense Presentation Guide

1. EXECUTIVE SUMMARY & CORE CONTRIBUTION

The 30-Second Elevator Pitch for Evaluators

Traditional Machine Learning Intrusion Detection Systems (ML-IDS) suffer from a critical vulnerability: **adversarial evasion attacks**. While defenses like feature perturbation and smoothing exist, static defenses force a dangerous trade-off between attack resistance and normal flow utility. Our platform introduces a closed-loop **Recall-Aware Feedback Controller (C1)** that continuously tracks real-time attack detection recall. When evasion attacks cause detection degradation, the controller dynamically shifts states (**Green** → **Yellow** → **Red**) and atomically modulates defense intensity to recover detection—achieving up to **+11.20% higher attack recall** with **0.00% False Positive Rate**.

2. CODEBASE ORGANIZATION & STREAMLINED ARCHITECTURE

To eliminate evaluation clutter, legacy prototype files (such as obsolete 58MB `models/` and scratch test scripts) have been pruned. The production platform is structured around **4 Primary Top-Level Scripts** supported by a modular `runtime_package/` library:

Primary Script	Role	Core Responsibility & Technical Execution
<code>server.py</code>	The Defense Server	FastAPI backend hosting the <code>SecurityEngine</code> singleton. Exposes endpoints (<code>/api/server/data</code> , <code>/api/server/state</code>), coordinates defense preprocessing, calls frozen RF model, updates the C1 controller, and streams telemetry via WebSocket.
<code>attacker_sim.py</code>	The Adversarial Client	Simulates real-world network traffic and launches 3 black-box evasion attacks (Silent Probing, Surrogate Transfer, Boundary Bisection) by streaming samples to <code>POST /api/server/data</code> .
<code>operator_benchmarks.py</code>	The Evaluator	Executes multi-batch controlled experiments (default 200 batches, 1000 flows) across Static vs. Recall-Aware arms. Computes recall, FPR, latency, and McNemar statistical significance.
<code>traffic_history.py</code>	Persistence Engine	SQLite database layer storing flow history, confusion matrix counts (TP, FP, TN, FN), and state logs for UI inspection and forensic verification.

What is `__init__.py` and Why is it in the Folders?

If an evaluator inspects folders like `runtime_package/attacks/__init__.py` or `runtime_package/defenses/__init__.py` and asks what it is:

- Python Package Marker:** In Python, `__init__.py` designates a folder as an importable module/package. It tells the Python interpreter that `runtime_package.attacks` can be imported using standard syntax: `from runtime_package.attacks.surrogate_transfer import SurrogateTransferAttack` .
- Why is it empty?** An empty `__init__.py` is PEP-standard best practice when a package does not require package-level side-effects or namespace pollution upon import. It keeps imports explicit, fast, and clean.

What to say to the panel: "In Python, `__init__.py` declares the directory as an importable package module. Keeping it empty is standard software engineering practice to prevent implicit side-effects while allowing clean, modular imports across our backend and benchmark runners."

3. END-TO-END REQUEST LIFECYCLE PIPELINE

When an incoming network flow arrives at `POST /api/server/data` , execution proceeds synchronously through 6 stages:

```
Client (attacker_sim.py / Real Flow)
| HTTP POST {features: [78 floats], sample_id: X, is_attack: True}
|
v
[server.py] endpoint `/api/server/data`
|
|→ 1. Feature Extraction: Deserializes 78-dimensional numpy float32 vector
|
|→ 2. Defense Pipeline:
|   If AFP → engine.defenses["afp"].defend(vector, intensity, ...)
|   If RS  → engine.defenses["rs"].predict_ensemble(vector, sigma, ...)
|   If FS  → engine.defenses["fs"].defend(vector, intensity, ...)
|   If None → Vector passes through unchanged (control baseline)
|
|→ 3. Classifier Inference:
|   Calls engine.model.predict_proba(defended_vector) on frozen_rf.joblib
```

- 4. Policy Verdict:
 - If `attack_probability` ≥ 0.50 → Block flow (HTTP 403 Forbidden)
 - If `attack_probability` < 0.50 → Allow flow (HTTP 200 OK)
- 5. Controller Feedback Loop:
 - Calls `engine.update_metrics_and_controller(ground_truth, pred, used_intensity)`
 - Updates confusion matrix counters: TP, FP, FN, TN
 - Every 5 attack decisions: evaluates rolling recall window and updates intensity
- 6. Real-Time Telemetry:
 - Broadcasts JSON event over WebSocket (`ws_manager.broadcast`) to `index.html` dashboard

4. RUNTIME PACKAGE DEEP DIVE (`runtime\_package/`)

All core algorithmic logic, defense math, attack strategies, and partitioning rules reside inside `runtime_package/`:

A. Dataset Role Partitioning

`runtime_package/data_loader.py` — Strict Role Separation & Zero Data Leakage

What it does: Loads the 90,000-row CIC-IDS dataset, verifies its SHA-256 fingerprint (`ffbb10a021aa...`), and strictly partitions data into two disjoint pools:

- `measurement_rows` (**72,000 flows**): Reserved exclusively as evaluation target flows for attack simulations and benchmark runs.
- `crafting_rows` (**18,000 flows**): Reserved exclusively for attackers to query the oracle when fitting surrogate models or finding boundaries.

What to say to the panel: "This file prevents data leakage. Adversaries are only permitted to query from the crafting pool, while evaluation targets are drawn exclusively from the measurement pool. The two sets are completely disjoint."

B. Defense Suite (`runtime\_package/defenses/`)

`defenses/base.py` — Protocol Immutability & Bounds Projection

What it does: The abstract base class inherited by all defenses. Enforces protocol immutability and physical value boundaries.

- Feature Mask Check:** Verifies that exactly **63 features are modifiable** and **15 features are protected** (TCP flags, ports, header lengths).
- `project_to_bounds(X_cand, X_orig)`: (1) Restores all 15 protected features back to their original values. (2) Clamps the 63 modifiable features strictly between `[train_min, train_max]` from `training_bounds.parquet`. (3) Emits `DefenseResult` with diagnostic counts.

What to say to the panel: "Every defense inherits from BaseDefense. It guarantees no defense ever modifies protocol constants or creates impossible packet numbers outside empirical training bounds."

`defenses/afp.py` — Adaptive Feature Poisoning

What it does: Adds variance-scaled bounded noise across modifiable features to disrupt adversarial gradients.

- Under the hood:** Loads empirical feature standard deviations (σ_{feat}) from `afp_benign_profile.parquet`. Computes perturbation: $\Delta = \alpha \times \text{intensity} \times \sigma_{\text{feat}} \times \text{sign}(\text{noise})$, zeroes out protected features, adds Δ to modifiable features, and calls `project_to_bounds()` ($\epsilon = 0.00030$, $\alpha = 0.50$).

What to say to the panel: "AFP adds variance-scaled bounded noise only to modifiable features. This breaks the fragile adversarial gradients of crafted attacks while preserving the statistical distribution of normal traffic."

`defenses/randomized_smoothing.py` — Ensemble Smoothing with Batched Vectorization

What it does: Implements randomized smoothing using an 11-member Gaussian ensemble ($\sigma = 0.00020$).

- Under the hood:** Generates 11 Gaussian-perturbed variants using deterministic seeds, stacks them into a single batched `(11, 78)` array, and executes classifier prediction in a **single vectorized C-level call (11x speedup, ~31ms latency)** before taking a majority vote.

What to say to the panel: "Randomized Smoothing evaluates 11 noisy variants and majority votes. If an adversary tailored a delicate perturbation to fool the model, Gaussian smoothing flips it back to the true class across the majority of samples."

`defenses/feature_squeezing.py` — Bit-Depth Quantization

What it does: Reduces numerical precision via bit-depth and step quantization to collapse fine-grained perturbations.

- Under the hood:** Derives step size $S = (X_{\text{max}} - X_{\text{min}}) / 2^d$ for intensity $d = 2$. Quantizes continuous floats via step rounding: $X_{\text{squeezed}} = \text{round}(X / S) \times S$, and clamps to bounds.

What to say to the panel: "Feature Squeezing rounds continuous feature values to discrete intervals. Tiny adversarial tweaks that rely on high-precision decimals get rounded away."

C. Feedback Controller (`runtime\_package/controller/`)

`controller/recall_controller.py` — Recall-Aware C1 Feedback Controller

What it does: The core brain of the platform. Dynamically adjusts defense intensity based on observed detection recall over a rolling evaluation window.

- **Window Accounting:** Stores observed batch outcomes in a `collections.deque(maxlen=window_size)`. Calculates rolling recall: $Recall = \sum TP / (\sum TP + \sum FN)$.
- **State Transitions & Multipliers:**
 - **Red State (Recall < 0.85):** Applies `fast_decay` ($\times 0.40$) to rapidly drop intensity and restore baseline detection.
 - **Yellow State ($0.85 \leq Recall < 0.95$):** Applies `slow_decay` ($\times 0.90$) to gently find equilibrium.
 - **Green State (Recall ≥ 0.95):** Applies `growth_factor` ($\times 1.30$) to increase robustness.
- **Clamping & Timing:** Clamps intensity to `[intensity_min, intensity_max]`. Batch `t` is evaluated with intensity chosen *before* batch `t` labels are seen. Updates trigger atomically every 5 attack decisions.

***What to say to the panel:** "This file implements our core contribution. It tracks live detection recall in real-time. If an active attack causes evasions, the controller detects it, shifts into Yellow or Red, and dynamically lowers intensity so the IDS can catch the attack."*

D. Adversarial Attack Suite (`runtime\_package/attacks/`)

`attacks/base.py` & `attacks/oracle.py` — Adversarial Abstraction & Budget Control

What it does: `base.py` defines the abstract interface for all attacks. `oracle.py` connects black-box attackers to the target model and strictly enforces query budget accounting, preventing budget overruns.

`attacks/silent_probing.py` — Zero-Query Control Baseline

What it does: Submits raw, unmodified evaluation attack flows (0 query budget) to establish baseline oracle detection.

***What to say to the panel:** "Silent Probing is our control baseline. It evaluates natural malicious flows without iterative crafting to establish unperturbed detection rates."*

`attacks/surrogate_transfer.py` — Black-Box Model Extraction & Transfer

What it does: Simulates a sophisticated black-box adversary who trains a local surrogate model and transfers crafted adversarial examples.

- **Under the hood:** (1) Queries oracle on crafting samples and fits an unpruned `DecisionTreeClassifier`. (2) Traverses tree to extract paths leading to 'Benign' predictions. (3) Computes bounding box intervals for modifiable features and projects attack vectors into the nearest benign interval.

***What to say to the panel:** "This simulates an advanced black-box attacker. The attacker queries the server, builds a surrogate Decision Tree, and crafts perturbations that transfer to the real Random Forest."*

`attacks/boundary_attack.py` — 1D Bisection Search

What it does: Finds minimal adversarial perturbations using binary search along the boundary within a 50-query budget.

- **Under the hood:** Takes an attack sample and a benign reference from the crafting pool, performs a 1D bisection search along the connecting line ($x_{mid} = \alpha x_{attack} + (1 - \alpha) x_{benign}$), and queries the oracle to find the exact threshold flipping the classification from Blocked (403) to Allowed (200).

***What to say to the panel:** "The boundary attack performs an active binary search between a known benign sample and an attack sample to find the thinnest possible perturbation that flips the classification."*

E. Assets & Support Modules

Asset / Module	Description & Purpose
<code>geoip/resolver.py</code>	Resolves flow source IP addresses to latitude/longitude and countries using local <code>dbip-city-lite.mmdb</code> for the Leaflet threat map.
<code>model/frozen_rf.joblib</code>	Pre-trained 100-tree Random Forest classifier trained on 78 CIC-IDS features. Loaded into memory once at startup.
<code>model/training_bounds.parquet</code>	Empirical min/max boundaries for all 78 features across the training dataset used by <code>project_to_bounds()</code> .
<code>model/feature_mask.json</code>	Boolean mask designating the 63 modifiable features vs. 15 immutable protocol fields.
<code>run.py</code>	Standalone offline CLI runner enabling researchers to evaluate attack and defense combinations without the HTTP web server.

5. DEFENSE PRESENTATION PROTOCOL & KEY PANEL DEFENSE Q&A

Live Demonstration Protocol (For Evaluators)

1. **Step 1: Open Dashboard** — Navigate to `http://127.0.0.1:8000/`. Point out the Threat Map, Confusion Matrix, and current Controller State badge.
2. **Step 2: Trigger Attack Stream** — In Terminal 2, run `python attacker_sim.py --mode surrogate`. Show incoming adversarial evasion attempts arriving in real time.
3. **Step 3: Run Comparative Benchmark** — Run `python operator_benchmarks.py --mode compare`. Present the statistical summary proving higher recall under Recall-Aware control.

Q1: "How do you prevent data leakage between training, attacks, and evaluation?"

Answer: "Through strict role-based data partitioning enforced in `runtime_package/data_loader.py`. The 90,000-row dataset is partitioned into 72,000 measurement rows and 18,000 crafting rows. Adversarial attacks (surrogate training, boundary search) are strictly confined to querying the crafting pool. Benchmark evaluation targets are drawn exclusively from the measurement pool. The two sets are completely disjoint."

Q2: "Does perturbing feature values break real network protocols?"

Answer: "No. The system enforces a strict 63-modifiable / 15-immutable feature mask in `defenses/base.py`. Protocol-critical header fields—such as TCP flags, source/destination ports, and IP header lengths—are protected and never modified. Furthermore, all perturbations are projected and clamped to empirical feature bounds derived from training data."

Q3: "Does the dynamic defense increase false alarms on benign traffic?"

Answer: "No. Across all 200-batch evaluations (500 benign flows evaluated per arm), the False Positive Rate remained strictly at 0.00%. Because perturbations are variance-bounded and clamped, legitimate network patterns remain on the correct side of the Random Forest decision boundary."

Q4: "How does the C1 Recall-Aware controller dynamically adapt intensity?"

Answer: "The controller in `recall_controller.py` calculates rolling recall over a window of completed batches. It evaluates against two frozen thresholds: $R_{critical}$ (85%) and R_{min} (95%). In the Green state ($\geq 95\%$), intensity grows by 1.30x to maximize robustness. In the Yellow state ($\geq 85\%$), intensity slowly decays by 0.90x. In the Red state ($< 85\%$), intensity drops rapidly by 0.40x to restore baseline attack detection."

Q5: "Why did you vectorize Randomized Smoothing?"

Answer: "In production ML-IDS, line-rate latency is critical. A naive implementation running 11 sequential scikit-learn tree traversals took ~345ms per packet, which caused buffer buildup. By generating all 11 perturbed variants in numpy and passing the stacked `(11, 78)` array into a single vectorized inference call, we achieved an 11x speedup to ~31ms without altering mathematical voting parity."